

Building a Vietnamese DBpedia: Linked Open Data from Vietnamese Wikipedia

Course: Semantic Web

Topic 2: Build a DBpedia version for Vietnamese language

Group 23

Ha Thi Kim Phung	20242460M
Do Ngoc Trung	20232311M
Bui Duc Phan Hoang	20252004M
Do Hoang Dung	20241184M

Lecturer: Dr. Do Ba Lam

October 2026

Abstract

DBpedia turns the semi-structured content of Wikipedia into Linked Data that can be queried with SPARQL and joined with other datasets. This report describes a DBpedia-style knowledge base for Vietnamese built from Vietnamese Wikipedia and Wikidata for a closed but densely connected domain: Vietnamese football (players, clubs, national teams and stadiums), provinces and universities. We (i) define an OWL ontology, `vio:`, with 18 classes and 54 properties, each class aligned to a DBpedia Ontology ancestor; (ii) select 990 entities through the Wikidata Query Service and collect their Vietnamese articles through the MediaWiki API; (iii) transform abstracts, infoboxes, categories and redirects into 88,898 RDF triples with stable IRIs that the project server dereferences, validated by eight quality checks and expanded by OWL 2 RL reasoning to 131,233 triples; (iv) link 777 entities to English DBpedia and all 990 to Wikidata with `owl:sameAs`, described as VoID linksets; and (v) serve the data through a web application with a SPARQL editor, DBpedia-style resource pages, HTTP content negotiation and a question-answering tab in which a large language model writes the SPARQL query. Reasoning lets the data be queried with the DBpedia vocabulary (0 asserted `dbo:Person` instances become 659) and answers multi-hop questions such as “which clubs has a player played for” through a property chain. Checked against the live DBpedia endpoint, 733 of the 777 English links resolve to an article. The code, data and 47 automated tests are available at <https://github.com/PhungHaThiKim/vietnamese-dbpedia>.

Key words: DBpedia; Linked Open Data; Vietnamese Wikipedia; ontology; OWL 2 RL; SPARQL; Wikidata; knowledge graph question answering

1 Introduction

DBpedia extracts structured information from Wikipedia—infobox fields, abstracts, categories, redirects and links between articles—and publishes it as RDF, so that the content of an encyclopedia can be queried with SPARQL and joined with other datasets [1, 2]. The English edition sits at the centre of the Linked Open Data cloud, and language chapters publish the same kind of data for other Wikipedia editions [3]. Vietnamese Wikipedia articles carry the same semi-structured information: the infobox of a footballer lists his date of birth, birthplace and every club he played for, season by season. As wikitext, however, it cannot be queried. A question such as “*Which players born in Nghệ An have played for the national team, and for which club do they play now?*” requires reading and joining several articles by hand.

This project addresses Topic 2 of the course, *Build a DBpedia version for Vietnamese language*, which consists of five tasks: (1) define an ontology; (2) collect Vietnamese articles from Wikipedia; (3) transform the data into 4-star Linked Data; (4) establish links to the English DBpedia; and (5) provide an interface to query the data through a SPARQL endpoint or terminal.

Rather than extracting all of Vietnamese Wikipedia shallowly, we chose a closed domain and extracted it in depth: Vietnamese *football* (players, clubs, national teams, stadiums), *provinces*

Table 1: Tasks of the assignment and where they are implemented. Module paths are relative to `vidbpedia/`.

Task	Implementation	Output	Section
1. Ontology	<code>ontology/*.ttl</code> , merged by <code>kg/ontology.py</code>	18 classes, 54 properties, 549 triples	3.2
2. Collect articles	<code>crawl/wikidata_seeds.py</code> , <code>crawl/wiki_enrich.py</code>	990 entities with their Vietnamese articles	3.3
3. 4-star data	<code>crawl/build_rdf.py</code> , <code>kg/postprocess.py</code>	131,233 triples in Turtle and N-Triples	3.4, 3.6
4. Link to DBpedia	enwiki sitelinks of the Wikidata item	777 <code>owl:sameAs</code> <code>dbp:</code> links, VoID linksets	3.5
5. Query interface	<code>web/</code> : FastAPI and Gradio	Linked Data routes, SPARQL editor, question answering	4

and *universities*. Three properties of this domain motivated the choice. First, its entities refer to one another, so the graph supports multi-hop questions: `player` $\rightarrow$ `career station` $\rightarrow$ `club` $\rightarrow$ `home stadium` $\rightarrow$ `province`, `player` $\rightarrow$ `birth province`, `university` $\rightarrow$ `province`. Second, 777 of the 990 selected entities (78%) have an English Wikipedia article, so most of them can be linked to English DBpedia. Third, 956 of the 990 articles (97%) contain an infobox, the main source that DBpedia extracts. Entities are selected through Wikidata rather than through the category tree of Vietnamese Wikipedia, because categories are assigned by editors and are not used consistently.

The contributions of this work are:

- an OWL ontology, `vio:`, with 18 classes and 54 properties. Every class has a DBpedia Ontology ancestor, 38 properties are sub-properties of a `dbo:` property, and OWL 2 RL axioms (a property chain, inverses, restrictions and disjointness) are designed for reasoning (Section 3.2);
- a reproducible extraction pipeline of six commands. All network requests go through a local cache, and the raw downloads are committed, so the dataset can be rebuilt without Internet access (Sections 3.3–3.6);
- a dataset of 131,233 triples, of which 88,898 are asserted and 41,730 are inferred, that passes eight automated quality checks with no error or warning (Section 5);
- 777 `owl:sameAs` links to English DBpedia, verified against the live endpoint, and 990 to Wikidata (Sections 3.5 and 5.4);
- a server that dereferences every resource IRI with HTTP 303 and content negotiation, and a web interface with resource pages, a SPARQL editor and natural-language question answering (Section 4).

Table 1 maps each task of the assignment to the part of the system that implements it. Section 2 introduces DBpedia and the Linked Data principles we follow. Section 3 describes the ontology and the extraction pipeline, Section 4 the server and the query interfaces, and Section 5 evaluates the dataset. Section 6 discusses design decisions and limitations, and Section 7 concludes.

2 Background and Related Work

DBpedia. DBpedia [1, 2] extracts several datasets from every Wikipedia article: labels, short and long abstracts, page identifiers, images, categories, redirects, external links, geographic coordinates, interlanguage links and infobox data. Infobox data is published twice. The *raw* extraction turns every template parameter into a property of the generic `dbp:` namespace. The *mapping-based* extraction uses community-maintained mappings from templates to the DBpedia Ontology (`dbo:`), a hand-built schema of classes and typed properties, which yields cleaner data at lower coverage. Each resource receives an IRI derived from its article title, such as `http://dbpedia.org/resource/Hanoi`. Non-English chapters publish data from their own Wikipedia edition under their own namespace and link their resources to the English edition. Kontokostas et al. [3] describe the Greek chapter and the issues that internationalisation raises: IRIs in a non-Latin script, language-specific templates, and interlinking across editions. Our project follows the same model for Vietnamese. We reuse the dataset types of DBpedia and its `dbo:` vocabulary, and we add a small domain ontology where DBpedia lacks the terms we need.

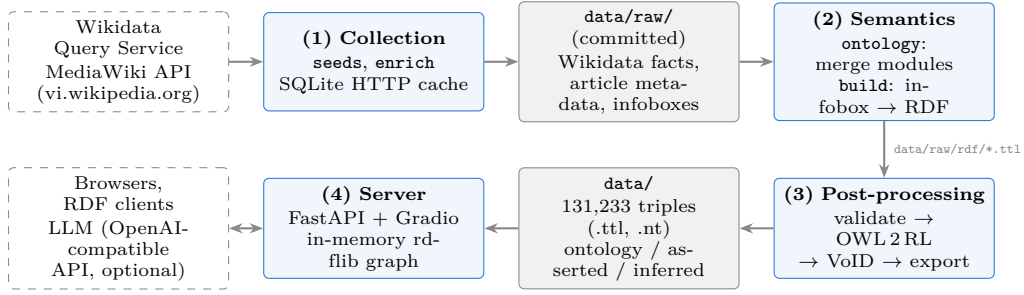


Figure 1: Architecture. Blue boxes are the four layers of the `vidbpedia` package, gray boxes are files on disk, dashed boxes are external systems. Only layer (1) calls the network.

Linked Data and the five-star scheme. The Linked Data principles ask publishers to name things with HTTP URIs, to return useful information in RDF when a URI is looked up, and to link to other URIs [4, 5]. The five-star scheme grades open data: one star for data on the web under an open licence, two for machine-readable structure, three for non-proprietary formats, four for using URIs to identify things, and five for linking to other data. Tasks 3 and 4 of the assignment correspond to the fourth and fifth stars. We follow the W3C recommendations for serving such data: *303 See Other* redirects with content negotiation, so that one URI identifies an entity and leads either to an HTML page or to an RDF document [6, 7], and a VoID description of the dataset and its linksets [8]. Data are expressed in RDF 1.1 [9] and queried with SPARQL 1.1 [10].

Wikidata. Wikidata [11] is a collaborative knowledge base whose items carry typed statements with qualifiers, and *sitelinks* to the article about the same item in every Wikipedia edition. The Wikidata Query Service exposes this data as a SPARQL endpoint [12]. We use Wikidata in three ways. It selects the entities through their class and country statements. It supplies precise dates and quantities, which are easy to parse in Wikidata but often written as free text in Vietnamese infoboxes. And its English sitelinks give the English DBpedia resource that corresponds to each entity.

Ontology engineering and reasoning. We designed the ontology from *competency questions*: questions the knowledge base must be able to answer, written before the schema and later used as tests [13]. To derive DBpedia types and shortcut relations from our own vocabulary, we use the OWL 2 RL profile [14]. Its rules can be applied by forward chaining in time polynomial in the size of the data, and the derived triples can be materialised and published alongside the asserted ones.

Question answering over knowledge graphs. Retrieval-augmented generation grounds the answer of a language model in retrieved evidence [15]. Over a knowledge graph the retrieval step can be a structured query: the model translates the question into SPARQL, the query runs on the graph, and the model answers from the result rows. Our question-answering tab follows this pattern. The model receives a summary of the schema and a few example queries, and invalid or empty queries are sent back to it for repair.

3 System Design

3.1 Architecture

The system is a Python package, `vidbpedia`, organised in four layers (Figure 1). Each step is one command, `python -m vidbpedia <step>`: `seeds`, `enrich`, `ontology`, `build`, `postprocess` and `serve`. Only the collection layer uses the network. Every HTTP response is stored in a SQLite cache, and the raw downloads are committed under `data/raw/`, so the later layers rebuild the full dataset offline and the result is identical between runs. Graphs are built with the `rdflib` API [16] rather than by concatenating Turtle strings, which rules out syntax and escaping errors. A single function creates every resource IRI, so an entity cannot receive two IRIs in two parts of the pipeline.

3.2 Ontology

The ontology is written as ten Turtle modules in `ontology/`, from `000-prefixes.ttl` to `420-football.ttl`. They are merged in numeric order into `vi-ontology.ttl` (549 triples) and

Table 2: Namespaces. The `vi.dbpedia.org` namespaces mirror the IRI scheme of DBpedia language chapters.

Prefix	Namespace	Used for
<code>vio:</code>	<code>http://vi.dbpedia.org/ontology/</code>	classes and properties of this project
<code>vres:</code>	<code>http://vi.dbpedia.org/resource/</code>	resources (entities)
<code>vip:</code>	<code>http://vi.dbpedia.org/property/</code>	raw infobox properties, like <code>dbp:</code>
<code>vcat:</code>	<code>http://vi.dbpedia.org/resource/Thể_loại:</code>	Wikipedia categories
<code>dbo:</code>	<code>http://dbpedia.org/ontology/</code>	DBpedia Ontology
<code>dbr:</code>	<code>http://dbpedia.org/resource/</code>	English DBpedia resources
<code>wd:</code>	<code>http://www.wikidata.org/entity/</code>	Wikidata items

Table 3: Class hierarchy of `vio:` (indentation shows `rdfs:subClassOf`), the DBpedia class each class is aligned to, and the number of instances before and after OWL 2 RL reasoning.

Class	DBpedia superclass	Asserted	After reasoning
Person	<code>dbo:Person</code>	0	659
Athlete	<code>dbo:Athlete</code>	0	611
FootballPlayer	<code>dbo:SoccerPlayer</code>	611	611
NationalTeamPlayer	—	0	486
Organisation	<code>dbo:Organisation</code>	0	410
FootballClub	<code>dbo:SoccerClub</code>	61	215
NationalFootballTeam	<code>dbo:NationalSoccerClub</code>	10	27
EducationalInstitution	<code>dbo:EducationalInstitution</code>	0	168
University	<code>dbo:University</code>	168	168
Location	<code>dbo:Place</code>	0	370
Province	<code>dbo:Province</code>	110	110
FormerProvince	—	76	76
Stadium	<code>dbo:Stadium</code>	29	44
Country	<code>dbo:Country</code>	1	1
CareerStation	<code>dbo:CareerStation</code>	0	3,382
YouthStation	—	527	527
ClubStation	—	1,966	1,966
NationalTeamStation	—	889	889

checked by `rdflib` when merged. Table 2 lists the namespaces. Following DBpedia [2], resources are named by their article title, infobox parameters have a raw namespace, and categories are SKOS concepts.

Classes. The 18 classes form four disjoint trees: people, organisations, locations and career stations (Table 3). Each class is a subclass of a DBpedia class, either directly or through its parent, so data described with `vio:` can also be queried with `dbo:` once the subclass axioms are applied (Section 3.6). We defined our own classes instead of using `dbo:` directly for two reasons. Some distinctions we need do not exist in DBpedia, such as `vio:FormerProvince` or the three kinds of career station. And keeping our classes separate means that our axioms constrain only our own data. `vio:FormerProvince` contains the provinces that were dissolved or merged, including the 2025 reorganisation of Vietnam’s provinces. A province is former if Wikidata records a dissolution date (P576), or if it no longer has a current P31 statement for “province of Vietnam” or “centrally governed city”. With this rule, the question “how many provinces are there now” returns the correct answer, 34, while history is kept through `vio:successor` and `vio:predecessor`.

Properties. The ontology has 54 properties, 26 object and 28 datatype properties. Each has a domain and range, labels in Vietnamese and English, and, when DBpedia has an equivalent of the same kind, an `rdfs:subPropertyOf` link to it (38 properties). Typical examples are `vio:birthDate` $\sqsubseteq$ `dbo:birthDate`, `vio:capacity` $\sqsubseteq$ `dbo:seatingCapacity`, and `vio:birthProvince` $\sqsubseteq$ `vio:birthPlace` $\sqsubseteq$ `dbo:birthPlace`. 22 datatype properties are functional, for example birth date, height and capacity. Some domains are left open on purpose. If `vio:latitude` or `vio:province` had the domain `Location`, every club and university that has coordinates or a province would be inferred to be a location, which contradicts the disjointness axioms. For the same reason `vio:ground` and `vio:manager` have the domain `Organisation` rather than `FootballClub`, because national teams also have a home ground and a coach.

Axioms. A career is modelled with the “role at a time” pattern of `dbo:CareerStation`. A player has one `vio:careerStation` per team and period, and the station carries the team, the start and end years, appearances, goals and an on-loan flag. Four kinds of axiom act on this structure during reasoning:

- a *property chain*, `vio:careerStation o vio:team  $\sqsubseteq$  vio:playedFor`, which gives a direct player–team edge;
- *inverses*, so that each relation can be queried in both directions: `vio:hasPlayer` is the inverse of `vio:playedFor`, and the same holds for the pairs `vio:hasPart/vio:isPartOf` and `vio:predecessor/vio:successor`;
- *restrictions*, which derive classes from the career data:

$$\begin{aligned} \exists \text{careerStation.NationalTeamStation} &\sqsubseteq \text{NationalTeamPlayer}, \\ \text{ClubStation} &\sqsubseteq \forall \text{team.FootballClub}. \end{aligned}$$

Similar $\forall$ restrictions hold for youth and national-team stations. They give a class to teams that are not seeds, such as the Japanese club Mito HollyHock;

- four *disjointness* axioms (`owl:AllDisjointClasses`): between the four roots, between club, national team and educational institution, between province, stadium and country, and between the three kinds of station. They let the reasoner detect modelling errors (Section 3.6).

3.3 Data Collection

Entity selection. `seeds` runs one Wikidata Query Service query per class. A query matches the class through `P31/P279*` and the country through `P17` or `P27` (Vietnam, `Q881`). It keeps only items with a Vietnamese Wikipedia article, and makes the English article optional. The query for players is shown below. Wikimedia list pages, disambiguation pages and titles starting with “Danh sách” (list of) are removed. When an item matches several classes, the first class in the priority order wins.

```
SELECT DISTINCT ?item ?viTitle ?enTitle WHERE {
  ?item wdt:P31 wd:Q5 ; wdt:P106 wd:Q937857 ; wdt:P27 wd:Q881 .    # human, footballer, Vietnam
  FILTER NOT EXISTS { ?item wdt:P31 wd:Q13406463 }                # Wikimedia list page
  FILTER NOT EXISTS { ?item wdt:P31 wd:Q4167410 }                  # disambiguation page
  ?vs schema:about ?item ; schema:isPartOf <https://vi.wikipedia.org/> ; schema:name ?viTitle .
  OPTIONAL { ?es schema:about ?item ; schema:isPartOf <https://en.wikipedia.org/> ;
             schema:name ?enTitle . } }
```

Facts are then fetched for blocks of at most 300 items, in three groups. Dates are fetched together with their precision, so that a year-only birth date becomes `xsd:gYear` instead of a fabricated 1 January. Quantities such as height, capacity, population, area and students are fetched together with their point in time (`P585`), so that the most recent population is kept. Relations are fetched together with their start, end, appearance and goal qualifiers. Birthplaces and locations are mapped to a province through `P131*`. Finally, labels and sitelinks are fetched for every referenced item. The result is 990 seeds: 611 players, 61 clubs, 10 national teams, 29 stadiums, 168 universities, 110 provinces and 1 country.

Article collection. `enrich` reads the Vietnamese articles of the seeds through the MediaWiki Action API, in batches of up to 50 titles, with `maxlag` set. For each article it stores the page ID, revision, length, coordinates and lead image; the plain-text introduction, used as the abstract; the non-hidden categories and incoming redirects; and, instead of the whole wikitext, only the infobox parameters and the URLs of the “Liên kết ngoài” (external links) section. The outermost infobox is found with `mwparsersfromhell`. Every title linked from an infobox is resolved, following redirects and flagging disambiguation pages: 2,588 titles, 71 of them disambiguation pages. Redirects to the infobox templates are collected as template aliases. Requests are spaced 2s apart for the Wikidata Query Service and 1s for Wikipedia, carry a User-Agent with contact information, and are retried on HTTP 429 and `maxlag` errors. The flags `--offline` and `--refresh` make the step use only the cache, or bypass it.

Table 4: DBpedia datasets reproduced for every entity, with their size in the current build.

Dataset	Properties	Size
Labels	<code>rdfs:label</code> @vi, @en	990 vi, 805 en
Abstracts	<code>dbo:abstract</code> (lead section), <code>rdfs:comment</code> (at most 400 characters)	990
Page metadata	<code>dbo:wikiPageID</code> , <code>dbo:wikiPageRevisionID</code> , <code>prov:wasDerivedFrom</code> , <code>foaf:isPrimaryTopicOf</code>	990
Images	<code>dbo:thumbnail</code> , <code>foaf:depiction</code>	543
Categories	<code>dct:subject</code> → <code>skos:Concept</code>	9,126 links, 1,154 categories
Redirects	<code>dbo:wikiPageRedirects</code> , <code>skos:altLabel</code>	1,274
External links	<code>dbo:wikiPageExternalLink</code> , <code>foaf:homepage</code>	713 links, 270 homepages
Coordinates	<code>vio:latitude/vio:longitude</code> ($\sqsubseteq$ <code>wgs84:</code>), <code>georss:point</code>	189 entities
Interlinks	<code>owl:sameAs</code> dbr:, <code>wd:</code>	777, 990
Raw infobox	<code>vip:key</code>	24,605 triples, 687 properties

3.4 Transformation to RDF

IRIs. A resource IRI is `vres:` followed by the Vietnamese title as MediaWiki normalises it: NFC Unicode, first character in upper case, spaces as underscores. Like DBpedia chapters, the IRI keeps Unicode characters, for example `vres:Nguyễn_Công_Phước`, and percent-encodes only the characters that are not allowed in an IRI. Career stations are numbered from youth to club to national team (`vres:Nguyễn_Công_Phước_3`). Redirects are separate resources that point to the main resource with `dbo:wikiPageRedirects`, and their titles are also added to the main resource as `skos:altLabel`, so that it can be found by its other names.

DBpedia datasets. For every entity, `build` produces the same basic datasets as DBpedia (Table 4). Raw infobox values are kept under `vip:`, as in DBpedia’s `dbp:` namespace.

Mapping-based infobox extraction. Infobox values are mapped to `vio:` properties in four stages.

1. *Render* the wikitext of each value. Common nested templates are interpreted instead of dropped: dates, heights, URLs and the list templates `ubl`, `plainlist` and `hlist`. References, comments and flag icons are removed.
2. *Parse* the value into a typed literal. The parsers understand Vietnamese number formats (`16.361,2`), open year ranges (`2015-`), loan markers (`→` and `(mượn)`), and accept heights only between 1.4 and 2.2 m.
3. *Map* infobox keys to `vio:` properties with the `MAPPINGS` table of five template families: players, clubs and national teams, stadiums, universities, and provinces. As in the mappings wiki of DBpedia, one property can have keys from both the Vietnamese and the English template.
4. *Choose a source* with the `PRECEDENCE` table when Wikidata and the infobox both have a value. Wikidata wins for dates, height, population, area, capacity, opening year, homepage and abbreviation. The infobox wins for shirt number, numbers of students and academic staff, motto and administrative code. For object properties, the values of both sources are combined.

Link targets are filtered by class: the team of a career station cannot be a province, a stadium or a university, and the `vio:ground` of a club must be a stadium.

Career stations come from the numbered parameters of the football biography infobox (`yearsn`, `clubsn`, `capsn` and `goalsn`) and their `youth` and `national` variants. For 594 of the 611 players the career is read from the infobox; the other 17 have no club rows (or no infobox) and fall back to the Wikidata statements `P54`. In total 601 players receive 3,382 stations. The build also writes `mapping_stats.json`, which counts mapped and unmapped keys for each template family, to guide which mappings to add next.

Table 5: Quality checks (`kg/validation.py`). The last build reports no error and no warning.

Check	Condition
<code>tbox</code>	sub-properties link properties of the same kind; every <code>vio:</code> class has a <code>dbo:</code> ancestor; every term has <code>vi</code> and <code>en</code> labels
<code>property_kinds</code>	object properties point to IRLs, datatype properties to literals
<code>datatype_ranges</code>	literals match the declared <code>rdfs:range</code> (including <code>rdf:langString</code>) and are well-formed
<code>required</code>	every entity has exactly one main class, a Vietnamese label, a source page and a Wikidata link
<code>functional</code>	functional properties have at most one value
<code>object_ranges</code>	if the target of an object property has a class, that class fits the range
<code>sameas</code>	no list or disambiguation targets; no two resources share a target
<code>plausibility</code>	coordinates inside Vietnam; birth year, capacity, population and height in a plausible range (warnings)

3.5 Interlinking

The English link of an entity comes from its Wikidata item: if the item has an English sitelink, the resource receives `owl:sameAs` to the `dbp:` resource of that title, written in the same Unicode IRI form that DBpedia uses. Using the sitelink of the same item avoids matching by label, which would be ambiguous for common Vietnamese names. Every entity also receives `owl:sameAs` to its Wikidata item. A validation check rejects links whose target is a list or disambiguation page, and links where two resources point to the same target. In addition to these instance links, the ontology is linked to DBpedia at the schema level through its subclass and sub-property axioms.

The dataset describes itself with VoID [8]. A `void:Dataset` carries the licence (CC BY-SA 4.0, as Wikipedia), the sources, the URI space, the vocabularies and the dump location. It has three subsets by provenance (ontology, asserted, inferred) with their triple counts, a class partition for each of the seven entity classes, and two `void:Linksets`, to DBpedia and to Wikidata, with the number of links.

3.6 Validation, Reasoning and Export

Validation. `postprocess` runs eight checks before reasoning (Table 5). Each check returns a list of issues with a severity, the check name, the subject and a detail. The same functions are run by the test suite, and with `--strict` a single error stops the pipeline.

Reasoning. Reasoning uses the OWL 2 RL rules of the `owlrl` library [14, 16] and runs in 117 s. Three choices keep it fast and correct.

1. Functional-property, cardinality and `owl:sameAs` axioms are removed from the TBox before reasoning. Otherwise OWL 2 RL would derive `x owl:sameAs x` for every node and merge entities that share a functional value. Functional properties are checked by validation instead.
2. Only the *skeleton* of the data is given to the reasoner: the `rdf:type` triples and the `vio:` triples of `vres:` resources. Abstracts, images and raw infobox triples cannot trigger any rule, so leaving them out changes nothing in the result.
3. Only new triples whose class or property is in `vio:`, `dbo:` or `wgs84:` are kept, and `owl:sameAs` is never kept. They are written to their own file, `inferred.nt`, so the interface can label each triple as asserted or inferred.

Reasoning also helped us find two modelling errors. First, it reported a violation of the disjointness between clubs and national teams. The cause was that `vio:ground`, `vio:manager` and `vio:chairman` had the domain `FootballClub`, so every national team with a home ground was inferred to be a club; the domain is now `Organisation`. Second, the question “which clubs have their home ground in Hà Nội” returned clubs from Đà Nẵng and Khánh Hòa. The cause was an axiom declaring `vio:tenant` the inverse of `vio:ground`, while the “tenants” field of the Hàng Đẫy stadium lists every team that ever played there. We removed the axiom, as DBpedia also keeps `dbo:tenant` and `dbo:ground` separate, and kept only tenants that are entities of the dataset. Figure 2 shows asserted and inferred triples around one player.

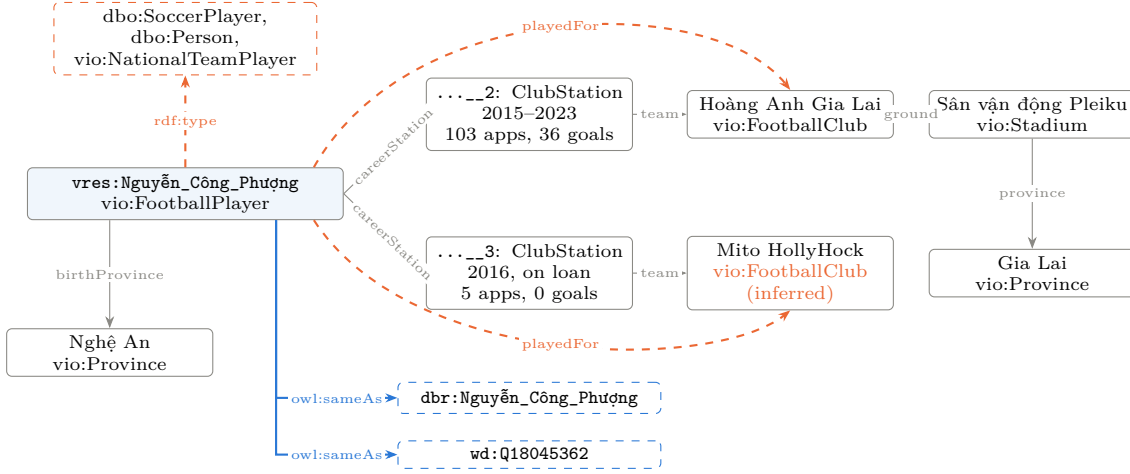


Figure 2: Two of the eleven career stations of Nguyễn Công Phượng. Gray edges are asserted, orange dashed edges are inferred by OWL 2 RL (`playedFor` through the property chain, the types through subclass axioms and the $\exists$ restriction, the class of Mito HollyHock through the $\forall$ restriction of `ClubStation`), and blue edges are links to external datasets. The inverse `vio:hasPlayer` edges are omitted.

Table 6: Linked Data routes (`web/linked_data.py`). An unknown name returns 404.

Route	Response
GET <code>/resource/{name}</code>	303 to <code>/?resource={name}</code> for browsers, or to <code>/data/{name}.ttl</code> , <code>.nt</code> , <code>.jsonld</code> or <code>.rdf</code> when <code>Accept</code> asks for Turtle, N-Triples, JSON-LD or RDF/XML; sends <code>Vary: Accept</code>
GET <code>/page/{name}</code>	303 to the HTML page, like <code>dbpedia.org/page/...</code>
GET <code>/data/{name}.{ext}</code>	every triple with the resource as subject, and up to 2,000 with it as object, in the requested format; sends <code>Access-Control-Allow-Origin: *</code>
GET <code>/ontology/{term}</code>	the definition of a <code>vio:</code> class or property, in Turtle

Export. The full dataset (ontology, asserted triples, inferred triples and the VoID description) is written as Turtle and N-Triples, and as RDF/XML with `--rdfxml`. The three parts are also written separately. The N-Triples file is read back and its triple count compared with the graph in memory. Statistics and the validation report are written as JSON.

4 Publishing and Querying

`python -m vidbpedia serve` starts one FastAPI application that hosts both the Linked Data routes and a Gradio [17] user interface. At start-up it loads the full N-Triples dump into an in-memory rdflib graph, loads `inferred.nt` into a set so that every triple can be labelled as asserted or inferred, builds a search index over labels and alternative names that ignores Vietnamese diacritics, and precomputes the class tree. The Linked Data routes are registered before Gradio is mounted at `/`, because the catch-all route of Gradio would otherwise shadow them.

4.1 Dereferenceable IRIs

The server follows the pattern of DBpedia and of the W3C Cool URIs note [6]. Looking up a resource IRI never returns content directly. It answers *303 See Other* and redirects either to an HTML page or to an RDF document, depending on the `Accept` header (Table 6). Because the interface is a single-page application, the HTML page is the address `/?resource=<name>`. When this address is loaded, the interface opens the resource tab on that entity, so every page has its own URL that can be shared.

Listing 1 shows an exchange recorded on the running server. As browsers do, clients must percent-encode the non-ASCII characters of the path.

In the response of Listing 1, the types `dbo:*` and the edge `vio:hasPlayer` are all inferred; `dbo:Animal` appears because the DBpedia Ontology places `dbo:Person` under `dbo:Animal`, and our alignment module (`050-dbo-alignment.ttl`) copies the DBpedia class chain.


```
$ curl -si -H "Accept: text/turtle" \
  http://127.0.0.1:7860/resource/Nguy%E1%BB%85n_C%C3%B4ng_Ph%C6%B0%E1%BB%A3ng
HTTP/1.1 303 See Other
vary: Accept
location: /data/Nguy%E1%BB%85n_C%C3%B4ng_Ph%C6%B0%E1%BB%A3ng.ttl

$ curl -sL -H "Accept: text/turtle" http://127.0.0.1:7860/resource/Nguy%E1%BB%85n...
vres:Mito_HollyHock vio:hasPlayer vres:Nguyễn_Công_Phượng .
...
vres:Nguyễn_Công_Phượng a dbo:Animal, dbo:Athlete, dbo:Person, dbo:SoccerPlayer, ... ;
  rdfs:label "Nguyễn Công Phượng"@en, "Nguyễn Công Phượng"@vi ;
  dbo:birthDate "1995-01-21"^^xsd:date ;
  ...
  vio:birthProvince vres:Nghệ_An ;
  vio:careerStation vres:Nguyễn_Công_Phượng__1, ... ;
  ...
  owl:sameAs dbr:Nguyễn_Công_Phượng, wd:Q18045362 ;
```

Listing 1: Dereferencing a resource IRI with curl. Response headers are abridged and the Turtle is an excerpt.

VỀ TÀI NGUYÊN

Nguyễn Công Phượng

Thực thể thuộc lớp Cầu thủ bóng đá - đó thì http://vi.dbpedia.org
http://vi.dbpedia.org/resource/Nguyễn_Công_Phượng

[Wikipedia tiếng Việt](#) /
 [DBpedia tiếng Anh](#) /
 [Wikidata Q18045362](#)

Cây phân lớp

Lớp khai báo và các lớp DBpedia suy ra bằng OWL 2 RL (rdfs:subClassOf).

```

dbo:Animal / suy luận
  dbo:Person / suy luận
    dbo:Athlete / suy luận
      dbo:SoccerPlayer / suy luận
        vio:Person Người / suy luận
          vio:Athlete Vận động viên
            suy luận ⊆ dbo:Athlete
              vio:FootballPlayer Cầu thủ bóng đá / khai báo ⊆
                dbo:SoccerPlayer
                  vio:NationalTeamPlayer Cầu thủ từng khoác áo đội tuyển quốc gia / suy luận

```

Liên kết dữ liệu mở

IRI
http://vi.dbpedia.org/resource/Nguyễn_Công_Phượng
 owl:sameAs dbr:Nguyễn_Công_Phượng /
 owl:sameAs wd:Q18045362 /
 foaf:isPrimaryTopicOf vi.wikipedia.org /
 prov:wasDerivedFrom bản sửa đổi 75557409 /

Tải RDF: Turtle · N-Triples · JSON-LD · RDF/XML

URI dereference được trên máy chủ này (303 + content negotiation):

```
curl -L -H "Accept: text/turtle" http://127.0.0.1:7860/resource/Nguy%E1%BB%85n_C%C3%B4ng_Ph%C6%B0%E1%BB%A3ng
```

Các prefix sau đã được khai báo sẵn, không cần viết lại: vio, vres, vip, vcat, dbo, dbr, wd, rdf, rdfs, owl, xsd, foaf, skos, dct, dc, prov, wgs84, georss, void.

Truy vấn

```

1 SELECT ?class (COUNT(DISTINCT ?x) AS ?n)
2 WHERE {
3   VALUES ?class {
4     dbo:Person dbo:SoccerPlayer
5     dbo:Organisation dbo:SoccerClub
6     dbo:University dbo:Place
7     dbo:Province dbo:Stadium
8     dbo:CareerStation }
9   ?x a ?class .
10 }
11 GROUP BY ?class ORDER BY DESC(?n)

```

Truy vấn mẫu

Đếm theo lớp DBpedia (suy luận) ▾

Định dạng kết quả

☒ Bảng
 ☐ JSON
 ☐ CSV

Chạy truy vấn

Xoá

Tìm thấy 9 kết quả

class	n
http://dbpedia.org/ontology/CareerStation	3382
http://dbpedia.org/ontology/Person	659
http://dbpedia.org/ontology/SoccerPlayer	611
http://dbpedia.org/ontology/Organisation	418
http://dbpedia.org/ontology/Place	378
http://dbpedia.org/ontology/SoccerClub	242
http://dbpedia.org/ontology/University	168
http://dbpedia.org/ontology/Province	118
http://dbpedia.org/ontology/Stadium	44

(a) Resource page (abstract and image hidden): class tree with asserted (*khai báo*) and inferred (*suy luận*) classes, and the Linked Data card.

(b) SPARQL tab: counting the instances of DBpedia classes, all of them inferred.

Figure 3: Web interface. Its labels are in Vietnamese.

4.2 Web Interface

The interface has four tabs, shown in Figure 3.

- **Resource tree** (the default tab) lists all 8,624 resources under the `vio:` class tree. Each resource appears under its most specific class only, like a file under its folder. Categories, redirects and resources that have only a label are listed in separate groups, and a filter box matches names with or without diacritics.
- **Resource** renders a page modelled on [dbpedia.org/page/...](http://dbpedia.org/page/) It shows the abstract, the image, and links to Wikipedia, DBpedia, Wikidata and OpenStreetMap. Below come the class tree with an asserted or inferred badge for each class, the Linked Data links with RDF downloads in four formats, an SVG neighbourhood graph with inferred edges drawn dashed, and a collapsible relation tree up to three hops deep (player → club → stadium → province). The page ends with the full property table and the incoming references.
- **Question answering** (Section 4.3).
- **SPARQL** is an editor where the 19 project prefixes are predeclared, with nine example queries. It shows the result as a table, or as SPARQL 1.1 JSON or CSV results.

Table 7: Entities by class, articles with an infobox, and entities linked to English DBpedia.

Class	Entities	With infobox	Linked to DBpedia
Football players	611	602	508
Football clubs	61	59	60
National teams	10	5	10
Stadiums	29	29	25
Universities	168	159	89
Provinces (34 current)	110	101	84
Country	1	1	1
Total	990	956 (97%)	777 (78%)

4.3 Natural-Language Question Answering

The question-answering tab lets users ask questions in Vietnamese. A language model writes a SPARQL query, the query runs on the graph, and the model answers from the result rows. The steps are:

1. The prompt contains a schema summary computed once from the data: for each main class and each kind of career station, its properties with their triple counts and an example value.
2. The prompt also gives rules and six example queries. The rules say to find entities by `rdfs:label` or `skos:altLabel` and never guess IRIs, to follow the career-station pattern or its `playedFor` shortcut, and to filter current provinces with `FILTER NOT EXISTS { ?p a vio:FormerProvince }`.
3. The generated query is parsed with `rdflib`, and only `SELECT` and `ASK` are accepted.
4. If the query fails or returns no rows, the error is sent back to the model for one repair.
5. A second prompt passes at most 30 result rows and asks for a JSON object with the reasoning steps and the answer. In *SPARQL mode* this call is skipped and the query and raw rows are shown instead.

The tab works with any OpenAI-compatible API (OpenAI, Google Gemini or a local Ollama model), selected in `.env`. The examples follow one performance rule. `rdflib` applies a `FILTER` only after joining the whole group, so the lookup of an entity by name is written as a subquery that runs first. For the career of Nguyễn Công Phượng this takes the query from 10.7s to 0.27s. The tests check the repair loop, SPARQL mode, the parsing of the answer and the rejection of write queries with a stubbed model, so they need no API key.

4.4 Standard SPARQL Endpoint

The built-in server offers SPARQL through the editor tab. It does not expose the SPARQL 1.1 Protocol [18], that is, a `/sparql` URL that programs can call over HTTP. For that use, the dump can be loaded into the Virtuoso triple store that is defined as an optional service in `docker-compose.yml` (port 8890), next to the application container (port 7860).

5 Evaluation

All figures in this section are computed from the published dataset by `report/report_numbers.py`. The link check of Section 5.4 queried the live DBpedia endpoint on 7 October 2026.

5.1 Dataset and Coverage

The dataset contains 131,233 triples: 88,898 asserted, 41,730 inferred, 549 in the ontology and 56 in the VoID description. They describe 990 entities, 3,382 career stations (527 youth, 1,966 club and 889 national-team stations), 1,154 categories and 1,274 redirects, for 8,624 distinct resource IRIs in total. 1,382 of these resources have a label but no class. They are targets of links in infobox values, such as cities, districts, people and competitions. Labels are in Vietnamese and English. Table 7 shows the coverage by class.

Table 8: Answers to the same query on the asserted data alone and on the published dataset (asserted plus inferred).

Query pattern	Asserted	Published	Inferred through
<code>?x a dbo:Person</code>	0	659	subclass, <code>rdfs:range</code>
<code>?x a dbo:SoccerPlayer</code>	0	611	<code>vio:FootballPlayer</code> $\sqsubseteq$ <code>dbo:SoccerPlayer</code>
<code>?x a dbo:Organisation</code>	0	410	subclass
<code>?x a dbo:Place</code>	0	370	subclass, <code>rdfs:range</code>
<code>?x vio:playedFor ?t</code>	0	2,772	chain <code>careerStation</code> $\circ$ <code>team</code>
<code>?t vio:hasPlayer ?x</code>	0	2,772	<code>owl:inverseOf</code>
<code>?x dbo:team ?t</code>	0	3,699	sub-property
<code>?x dbo:birthPlace ?p</code>	0	802	sub-property

Infobox coverage is high for every class except national teams, whose Vietnamese articles often have no infobox. Links to English DBpedia are complete for clubs and national teams but cover only about half of the universities, because many Vietnamese universities have no English article. Within the football domain, the main properties of players are well covered: position for 586 of 611 players, birth date for 572, height for 534, birth province for 462 and current club for 442. 56 of 61 clubs have a home ground and 142 of 168 universities are linked to a province. Of the 3,382 career stations, 129 (3.8%) have no `vio:team`, because the team is written as plain text in the infobox, or its link was rejected by the class filter.

5.2 Effect of Reasoning

Table 8 compares queries on the asserted data alone with the same queries on the published dataset. Of the 41,730 inferred triples, 18,341 are types and 23,389 are property values. Without reasoning, none of the DBpedia classes or properties has an instance: the data uses only the `vio:` vocabulary. After reasoning, a client that knows only DBpedia can ask for `dbo:SoccerPlayer` or `dbo:team` and get answers. The 48 people who are not players (coaches, chairmen and rectors) receive `dbo:Person` from the range of the property that links to them. The property chain turns a two-step path through career stations into 2,772 direct `vio:playedFor` edges. The class counts of Table 3 show the effect of the restrictions: 486 players become `vio:NationalTeamPlayer`, and 154 clubs that are not seeds, such as Mito HollyHock, become `vio:FootballClub` because they are the team of a club station.

5.3 Quality and Tests

The eight checks of Table 5 report no error and no warning on the last build, and the N-Triples dump reads back with the same number of triples. The test suite has 47 tests and passes in about 15s. There are 12 competency-question tests on the real data (Section 5.5); 3 quality tests, which run the validation and check that the inferred part contains no `owl:sameAs`; 3 reasoning tests of the chain, the inverse, the restriction and a disjointness violation on small graphs; 6 parser tests for nested templates, Vietnamese numbers, dates, loans and IRIs; 5 tests of the question-answering loop with a stubbed model; and 18 interface tests of the Linked Data routes (303 redirects, content negotiation, the four RDF formats read back by `rdflib`), the resource page and the resource tree. Tests use lower bounds instead of exact counts, so they keep passing when the data is crawled again.

5.4 Links to English DBpedia

We sent every `dbr:` target to the public DBpedia SPARQL endpoint and checked whether it describes an article, a redirect or nothing (Table 9). 733 of the 777 links (94.3%) point to an article. This also confirms that DBpedia uses the same Unicode IRIs that we generate. Twelve point to a resource that DBpedia records as a redirect (for example `dbr:Quảng_Ninh`). Thirty-two point to titles that DBpedia does not contain. Half of them (16) are clubs, several with recent titles such as `dbr:Becamex_Ho_Chi_Minh_City_FC` and `dbr:Ho_Chi_Minh_City_FC_(2025)`. In both cases our links come from today’s English Wikipedia titles, while DBpedia was extracted from an older snapshot of English Wikipedia.

Table 9: owl:sameAs targets in English DBpedia, checked against dbpedia.org/sparql on 7 October 2026.

Status of the dbr: target	Links	Share
Article	733	94.3%
Redirect (dbo:wikiPageRedirects)	12	1.5%
Not in DBpedia	32	4.1%
Total	777	100%

Table 10: Competency questions and the answers returned by the graph.

Question	Answer	Relies on
How many provinces and centrally governed cities are there now?	34 (of 110 provinces, 76 are former)	vio:FormerProvince
Which unit succeeded the former province of Hà Tây?	Hà Nội	vio:successor
Which teams has Nguyễn Công Phượng played for?	11 teams over 11 stations, 7 of them at clubs	property chain
Which provinces are home to the most players?	Nghệ An 58, Hà Nội 48, Thanh Hóa 42	vio:birthProvince
How many universities are in Hà Nội?	54	vio:province
How many clubs have a home ground in a known province?	44	club → stadium → province
Which entity is meant by “Công Phượng”?	Nguyễn Công Phượng	dbo:wikiPageRedirects
Which entities are DBpedia soccer players?	611	subclass reasoning

5.5 Competency Questions

Table 10 lists competency questions that guided the ontology, with the answers that the graph returns. Each question is a test in tests/test_sparql.py, and most need several hops or an inferred triple.

5.6 Five-Star Assessment

Table 11 assesses the dataset against the five-star scheme [4].

6 Discussion

Design decisions. Four decisions shaped the system.

- *Entities are selected through Wikidata, not through Vietnamese categories.* Categories are assigned inconsistently by editors, while Wikidata class and country statements give a precise selection and an English sitelink for interlinking.
- *The data uses our own vocabulary, aligned to DBpedia, rather than dbo: directly.* This let us model what DBpedia lacks, such as typed career stations and former provinces, while reasoning still answers queries written in the DBpedia vocabulary.
- *Inferred triples are materialised and stored separately.* Clients get DBpedia types without running a reasoner, and the interface can show which statements were asserted and which were inferred.
- *The serving stack is a Python application with an in-memory rdflib graph.* A graph of 131 thousand triples loads in a few seconds, and one process serves both the Linked Data routes and the interface. A dedicated triple store remains optional.

Limitations. The current system has six limitations.

Table 11: The dataset against the five-star scheme.

Star	Evidence
★ open licence on the web	CC BY-SA 4.0, declared in the VoID description and the ontology header; dumps published in a Git repository
★★ machine-readable	RDF triples, not text or tables
★★★ open formats	Turtle and N-Triples dumps; each resource also as JSON-LD and RDF/XML
★★★★ URIs for things	an HTTP IRI for every entity, career station, category and redirect; dereferenced with 303 and content negotiation by the project server (Section 4.1)
★★★★★ linked	777 <code>owl:sameAs</code> links to English DBpedia, 990 to Wikidata, described as VoID linksets; classes and properties aligned to <code>dbo:</code>

- *Coverage.* The dataset covers 990 entities of one domain, not the whole of Vietnamese Wikipedia. The pipeline is generic, but each new class needs a seed query, ontology terms, an infobox mapping and a build function.
- *Resolvable IRIs.* The `vi.dbpedia.org` namespace follows the convention of DBpedia chapters, but the project does not control that domain. The IRIs are therefore dereferenceable only through a running project server, not on the public Web. Publishing on a domain we control, or under a persistent identifier service such as `w3id.org`, would make them resolvable.
- *Link drift.* 44 of the 777 English links do not reach a DBpedia article (Section 5.4). Following `dbo:wikiPageRedirects` would fix the twelve redirects. The other 32 should be checked again after each DBpedia release.
- *Data gaps.* 129 career stations have no team IRI, 26 universities have no province, and the DBpedia datasets of page links (`dbo:wikiPageWikiLink`) and of the category hierarchy (`skos:broader`) are not extracted.
- *Serving.* The server has no SPARQL 1.1 Protocol endpoint and no time limit on queries. `rdflib` also executes `SERVICE` clauses, which send requests to other endpoints. All three should be addressed before the server is exposed on a public network.
- *Question answering.* The quality of the question-answering tab depends on the language model, and we did not measure its accuracy on a benchmark of questions.

7 Conclusion

We built a DBpedia-style knowledge base for Vietnamese football, provinces and universities, covering the five tasks of the assignment. An OWL ontology of 18 classes and 54 properties is aligned to the DBpedia Ontology. 990 entities were selected through Wikidata and their Vietnamese articles collected through the MediaWiki API. Abstracts, infoboxes, categories and redirects were transformed into 88,898 validated triples with stable IRIs, which OWL 2 RL reasoning extends to 131,233. 777 entities are linked to English DBpedia, of which 733 resolve to an article today, and all 990 to Wikidata. A server dereferences the IRIs with content negotiation and offers resource pages, a SPARQL editor and natural-language question answering. Reasoning is what makes the data interoperable: it adds the DBpedia types and properties that DBpedia clients expect, and the shortcut edges that turn multi-hop questions into simple queries.

Future work follows from the limitations: a standard SPARQL Protocol endpoint with query time limits; more classes, such as coaches and competitions, together with page links; a link check against DBpedia at every build; and a benchmark of Vietnamese questions to evaluate the question-answering tab.

References

- [1] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, “DBpedia: A nucleus for a web of open data,” in *The Semantic Web: 6th International Semantic Web Conference (ISWC 2007) and 2nd Asian Semantic Web Conference (ASWC 2007)*, ser. Lecture Notes in Computer Science, vol. 4825. Springer, 2007, pp. 722–735.
- [2] J. Lehmann, R. Isele, M. Jakob, A. Jentzsch, D. Kontokostas, P. N. Mendes, S. Hellmann, M. Morsey, P. van Kleef, S. Auer, and C. Bizer, “DBpedia – a large-scale, multilingual knowledge base extracted from Wikipedia,” *Semantic Web*, vol. 6, no. 2, pp. 167–195, 2015.
- [3] D. Kontokostas, C. Bratsas, S. Auer, S. Hellmann, I. Antoniou, and G. Metakides, “Internationalization of linked data: The case of the Greek DBpedia edition,” *Journal of Web Semantics*, vol. 15, pp. 51–61, 2012.
- [4] T. Berners-Lee, “Linked data – design issues,” W3C Design Issues, 2006, the five-star scheme was added in 2010. [Online]. Available: <https://www.w3.org/DesignIssues/LinkedData.html>
- [5] T. Heath and C. Bizer, *Linked Data: Evolving the Web into a Global Data Space*, ser. Synthesis Lectures on the Semantic Web: Theory and Technology. Morgan & Claypool, 2011.
- [6] L. Sauermann and R. Cyganiak, “Cool URIs for the semantic web,” W3C Interest Group Note, 2008. [Online]. Available: <https://www.w3.org/TR/cooluris/>
- [7] B. Hyland, G. Atemez, and B. Villazón-Terrazas, “Best practices for publishing linked data,” W3C Working Group Note, 2014. [Online]. Available: <https://www.w3.org/TR/ld-bp/>
- [8] K. Alexander, R. Cyganiak, M. Hausenblas, and J. Zhao, “Describing linked datasets with the VoID vocabulary,” W3C Interest Group Note, 2011. [Online]. Available: <https://www.w3.org/TR/void/>
- [9] R. Cyganiak, D. Wood, and M. Lanthaler, “RDF 1.1 concepts and abstract syntax,” W3C Recommendation, 2014. [Online]. Available: <https://www.w3.org/TR/rdf11-concepts/>
- [10] S. Harris and A. Seaborne, “SPARQL 1.1 query language,” W3C Recommendation, 2013. [Online]. Available: <https://www.w3.org/TR/sparql11-query/>
- [11] D. Vrandečić and M. Krötzsch, “Wikidata: a free collaborative knowledgebase,” *Communications of the ACM*, vol. 57, no. 10, pp. 78–85, 2014.
- [12] S. Malyshev, M. Krötzsch, L. González, J. Gonsior, and A. Bielefeldt, “Getting the most out of Wikidata: Semantic technology usage in Wikipedia’s knowledge graph,” in *The Semantic Web – ISWC 2018*, ser. Lecture Notes in Computer Science, vol. 11137. Springer, 2018, pp. 376–394.
- [13] M. Grüninger and M. S. Fox, “Methodology for the design and evaluation of ontologies,” in *Proceedings of the IJCAI-95 Workshop on Basic Ontological Issues in Knowledge Sharing*, Montreal, Canada, 1995.
- [14] B. Motik, B. Cuenca Grau, I. Horrocks, Z. Wu, A. Fokoue, and C. Lutz, “OWL 2 web ontology language profiles (second edition),” W3C Recommendation, 2012. [Online]. Available: <https://www.w3.org/TR/owl2-profiles/>
- [15] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.
- [16] RDFLib Team, “RDFLib and OWL-RL: Python libraries for RDF and OWL 2 RL reasoning,” Software, versions 7.6.0 and 7.6.2, accessed October 2026. [Online]. Available: <https://github.com/RDFLib>
- [17] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, “Gradio: Hassle-free sharing and testing of ML models in the wild,” arXiv preprint arXiv:1906.02569, 2019. [Online]. Available: <https://arxiv.org/abs/1906.02569>
- [18] L. Feigenbaum, G. T. Williams, K. G. Clark, and E. Torres, “SPARQL 1.1 protocol,” W3C Recommendation, 2013. [Online]. Available: <https://www.w3.org/TR/sparql11-protocol/>